

# Домашнее задание №6

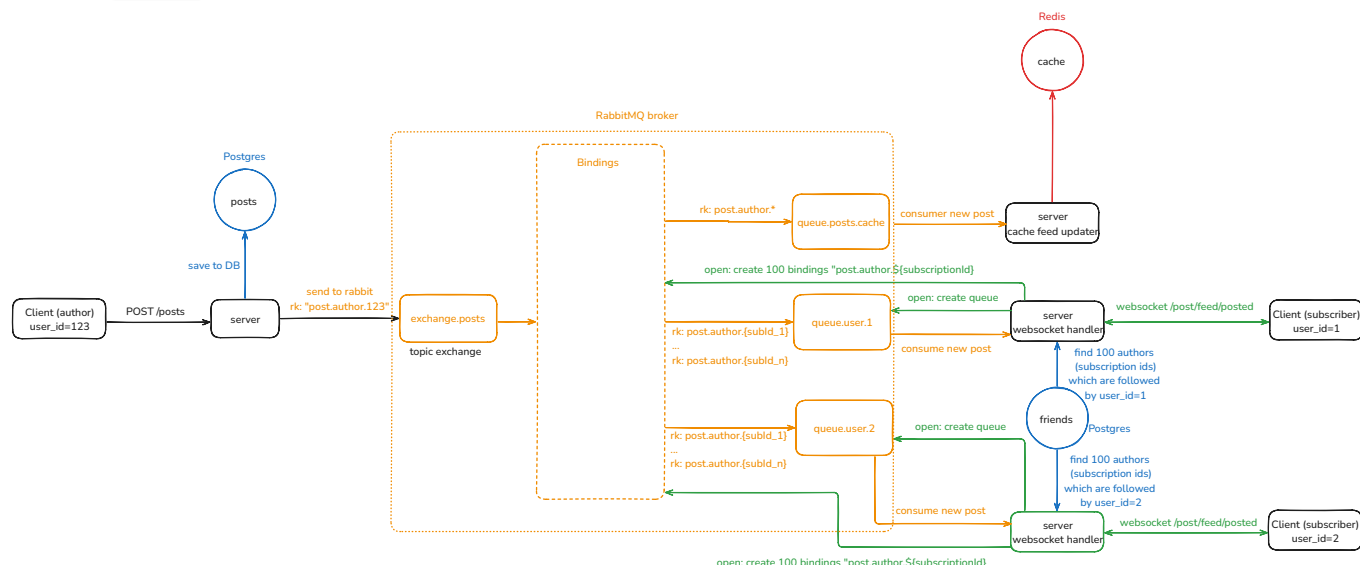
Разработать WebSocket сервер, при помощи которого подключенные клиенты будут сразу получать обновления постов своих друзей. При добавлении нового поста друга подписчику websocket'a должно приходить событие о новом посте, тем самым обеспечивая обновление ленты в реальном времени.

## Требования

- Формирование лент работает через очередь (отложено)
- При реализации обязательно обеспечить отправку только целевым пользователям это событие (можно применить Routing Key из RabbitMQ)

## Схема приложения

Для реализации требований я использовал `WebSocketHandler` и брокер сообщений RabbitMQ. При запуске приложения автоматически создаётся exchange `exchange.posts` с типом `topic`.



## 1—Подключение клиента к вебсокету

Правая часть схемы. Что происходит при подключении клиента к вебсокету:

- открывается вебсокет-сессия
- в RabbitMQ создаётся очередь `queue.user.${userId}`, где `${userId}` — id клиента, подключившегося к вебсокету

- в Postgres в таблице `friends` ищутся id максимум 100 пользователей, на которых подписан клиент – назовём их `subscription ids`
- для каждого `subscriptionId` в RabbitMQ создаётся binding между exchange `exchange.posts` и созданной очередью `queue.user.${userId}`; также настраивается `routing_key=post.author.${subscriptionId}`
- в канале подключения к RabbitMQ запускается консьюмер новой очереди, где в `deliver callback` выполняется отправка полученного из очереди сообщения в вебсокет-сессию

Теперь вебсокет готов к обслуживанию клиента. Таким образом на каждого клиента создаётся:

- 1 вебсокет-сессия
- 1 очередь
- от 0 до 100 bindings

## Заккрытие вебсокет-сессии

При закрытии сессии удаляется очередь в RabbitMQ, а вместе с ней и все `bindings`.  
Дополнительных действий не требуется.

## 2—Создание нового поста автором

Левая часть схемы. Автор создаёт новый пост с помощью эндпоинта `POST /posts`:

- сервер сохраняет новый пост в Postgres,
- затем асинхронно отправляет сообщение в RabbitMQ:
  - exchange `exchange.posts`
  - `routing_key=post.author.${authorId}`, где `${authorId}` – id автора, который создал новый пост

На этом синхронный вызов завершается.

Стоит отметить, что отправка сообщения в RabbitMQ выполняется отдельным классом `MQService` асинхронно в отдельном треде. Я это сделал для того, чтобы возможные проблемы при отправке в RabbitMQ не помешали сохранить пост.

## 3—Формирование лент (обновление кэша) через очередь

Правая верхняя часть схемы. Метод обновление кэшей лент и постов в классе `PostCacheService` не поменялся: сервер ищет подписчиков автора поста, для каждого подписчика обновляет кэш ленты, а затем кэширует сам новый пост.

Раньше этот метод явно вызывался сразу после сохранения поста в `PostService`. Теперь этот метод слушает очередь `queue.posts.cache`. Для этой очереди настроен `binding` с `routing_key=post.author.*` – это значит, что все новые посты абсолютно любого автора будут прилетать в эту очередь.

## Вывод

В данной работе удалось настроить вебсокет-сервер, который позволяет клиентам получать новые посты тех авторов, на которых клиенты подписаны. Асинхронная схема работы на базе брокера сообщений RabbitMQ позволяет настраивать и масштабировать эту часть системы, что является важным плюсом такой архитектуры.